

Alfresco LDAP Authentication & Synchronization Setup

Reproducible OpenLDAP test environment for ACS via Docker Compose

Based on: `docker-compose/26.N-compose-governance.yaml`,
`docker-compose/ldap-custom-ldif/50-users.ldif`,
`docker-compose/behaviour-derive-names-context.xml`, `docker-compose/derive-names-onCreateNode.js`
Location: `C:\AlfrescoWork\acs-deployment\docker-compose`

This guide documents, end-to-end, how an OpenLDAP directory server is stood up alongside an Alfresco Governance Services container, how Alfresco is configured to authenticate against and synchronize users/groups from it, how a custom LDIF file seeds edge-case test users, and how a small repository customization patches up users that are missing a first or last name. Follow the steps here to reproduce the whole environment from a clean checkout.

1. Overview & Architecture

The LDAP test setup lives entirely inside one Docker Compose stack file, `26.N-compose-governance.yaml`. It adds a single new service — **openldap** — to the normal ACS/Governance stack, and wires the **alfresco** service's authentication chain to talk to it over plain LDAP (port 389, no TLS). The two containers communicate over the Compose-managed Docker network using the service name `openldap` as the hostname.

Component	Role
openldap (osixia/openldap:1.5.0)	LDAP directory server holding users/groups; seeded with test data on first boot
alfresco (governance repository)	Authenticates users and synchronizes <code>cm:person/cm:authority</code> nodes from LDAP
50-users.ldif (custom bootstrap file)	Pre-loads 3 test users covering missing first-name/last-name scenarios
behaviour-derive-names-context.xml + derive-names-post-sync-context.xml + RepositoryUtils.jsp	RepositoryUtils.jsp: derives a blank firstName/lastName from the other, post-sync

Alfresco's authentication chain is set to `ldap1:ldap,alfrescoNTLm:alfrescoNTLm` — meaning a login attempt is first tried against the LDAP subsystem (bind as the user against OpenLDAP); if that fails, it falls back to the repository's own (NTLM/DB-backed) authentication, so the built-in **admin** user still works even though it is not an LDAP entry.

2. The OpenLDAP Container

Defined as a new `openldap` service in the compose file, built from the public `osixia/openldap:1.5.0` image with one customization layered on top: the custom seed LDIF is copied into the image's bootstrap folder so it is loaded automatically the first time the container initializes its database.

```
openldap:
  build:
    context: .
    dockerfile_inline: |
      FROM osixia/openldap:1.5.0
      COPY ldap-custom-ldif/50-users.ldif /container/service/slapd/assets/config/bootstrap/ldif/custom/50-users.
  mem_limit: 256m
  environment:
    LDAP_ORGANISATION: "Example Inc"
    LDAP_DOMAIN: "example.org"
    LDAP_BASE_DN: "dc=example,dc=org"
    LDAP_ADMIN_PASSWORD: "admin"
    LDAP_TLS: "false"
  ports:
    - "389:389"
  healthcheck:
    test: ["CMD", "ldapsearch", "-x", "-H", "ldap://localhost:389",
      "-D", "cn=admin,dc=example,dc=org", "-w", "admin",
      "-b", "dc=example,dc=org"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s
```

Key points

- **Base DN** is `dc=example,dc=org` (derived from `LDAP_DOMAIN=example.org`).

- **Admin bind DN** is `cn=admin,dc=example,dc=org` with password `admin` — used by Alfresco's synchronization subsystem (it needs an account that can search/read the whole tree), not by end users.
- **Port 389** (plain LDAP, no TLS) is published to the host as well, so the directory can be queried directly from the host machine with tools like `ldapsearch` or Apache Directory Studio for troubleshooting.
- The **healthcheck** performs an authenticated search against the base DN; Alfresco's own `depends_on: openldap: condition: service_healthy` ensures the repository container doesn't start trying to synchronize before the directory is actually ready to answer queries.
- Custom bootstrap LDIFs placed under `/container/service/slapd/assets/config/bootstrap/ldif/custom/` are only applied the **first time** the container's data volume is initialized. If you change the LDIF and need it reloaded, you must remove the container's underlying volume/data first (see Troubleshooting).

3. Seed Data — 50-users.ldif

Location: `docker-compose/ldap-custom-ldif/50-users.ldif`. This file creates an organizational unit for users and three test accounts that deliberately exercise the "missing name attribute" edge cases that LDAP/Active Directory synchronization can run into in the real world.

```
dn: ou=users,dc=example,dc=org
objectClass: organizationalUnit
ou: users

# Scenario 1: ONLY FIRST NAME (givenName present, sn effectively blank)
dn: uid=jonly,ou=users,dc=example,dc=org
objectClass: inetOrgPerson
uid: jonly
cn: Jon
givenName: Jon
sn:: IA== # base64 for a single space character
mail: jonly@example.org
userPassword: password123

# Scenario 2: ONLY LAST NAME (sn present, no givenName attribute at all)
dn: uid=smithonly,ou=users,dc=example,dc=org
objectClass: inetOrgPerson
uid: smithonly
cn: Smith
sn: Smith
mail: smithonly@example.org
userPassword: password123

# Scenario 3: BOTH first and last name present
dn: uid=jdoe,ou=users,dc=example,dc=org
objectClass: inetOrgPerson
uid: jdoe
cn: Jane Doe
givenName: Jane
sn: Doe
mail: jdoe@example.org
userPassword: password123
```

Username	givenName	sn	Purpose
jonly	Jon	(single space)	Simulates AD user with a blank Last Name field
smithonly	(absent)	Smith	Simulates user with no First Name attribute at all
jdoe	Jane	Doe	Normal/control case — both names present

All three users share the password `password123`. Note: `sn` (surname) is a **mandatory** attribute in the standard LDAP person schema that `inetOrgPerson` extends, so it can never be truly absent the way it can in Active Directory. To faithfully emulate an AD user whose admin left "Last Name" blank, the `jonly` entry sets `sn` to a single space character — encoded as base64 (`IA==`) because LDIF values that start with whitespace must be base64-encoded.

4. Alfresco Authentication & Synchronization Config

All settings are passed to the `alfresco` container as `-D` system properties inside `JAVA_OPTS` (equivalent to setting them in `alfresco-global.properties`):

```
-Dauthentication.chain=ldap1:ldap,alfrescoNtlm:alfrescoNtlm
```

```
# --- Authentication (used at login time to bind as the user) ---
-Dldap.authentication.active=true
-Dldap.authentication.userNameFormat=uid=%s,ou=users,dc=example,dc=org
-Dldap.authentication.java.naming.factory.initial=com.sun.jndi.ldap.LdapCtxFactory
-Dldap.authentication.java.naming.provider.url=ldap://openldap:389
-Dldap.authentication.java.naming.security.authentication=simple

# --- Synchronization (scheduled job that imports users/groups) ---
-Dldap.synchronization.active=true
-Dldap.synchronization.java.naming.factory.initial=com.sun.jndi.ldap.LdapCtxFactory
-Dldap.synchronization.java.naming.provider.url=ldap://openldap:389
-Dldap.synchronization.java.naming.security.authentication=simple
-Dldap.synchronization.java.naming.security.principal=cn=admin,dc=example,dc=org
-Dldap.synchronization.java.naming.security.credentials=admin
-Dldap.synchronization.userSearchBase=ou=users,dc=example,dc=org
-Dldap.synchronization.groupSearchBase=dc=example,dc=org
-Dldap.synchronization.userIdAttributeName=uid
-Dldap.synchronization.userFirstNameAttributeName=givenName
-Dldap.synchronization.userLastNameAttributeName=sn
-Dldap.synchronization.userEmailAttributeName=mail
-Dldap.synchronization.defaultHomeFolderProvider=largeHomeFolderProvider
-Dldap.synchronization.groupIdAttributeName=cn
-Dldap.synchronization.groupType=groupOfNames
-Dldap.synchronization.personType=inetOrgPerson
-Dldap.synchronization.groupMemberAttributeName=member
```

Authentication vs. Synchronization

These are two independent subsystems that happen to point at the same directory:

- **ldap.authentication.*** runs at login time. Alfresco takes the username the user typed, substitutes it into `userNameFormat` (`uid=%s,ou=users,dc=example,dc=org`) to build a full DN, and attempts an LDAP *simple bind* with that DN and the password the user typed. Success = authenticated.
- **ldap.synchronization.*** runs as a background scheduled job (and on repository startup) using the admin service account (`cn=admin,dc=example,dc=org`) to search the whole directory and create/update corresponding `cm:person` (user) and `cm:authority` (group) nodes in the Alfresco repository, mapping LDAP attributes (`givenName`, `sn`, `mail`, ...) onto Alfresco person properties.
- A user can only successfully log in via LDAP *and* see a proper profile in Share if **both** are configured consistently against the same base DN and attribute names.

5. Repository Customization: Deriving Missing Names

`cm:firstName` is declared mandatory in the Alfresco content model, but only as a *soft* (forms/UI) constraint — it is not enforced=`"true"`. LDAP synchronization writes nodes directly via `NodeService`, bypassing form validation entirely, so a user synced from a directory entry with a blank `givenName` or `sn` ends up with a genuinely empty name field in Share. There is no built-in `Ldap.synchronization.*` property that supplies a fallback value for a missing attribute, so this environment adds a small custom behaviour to patch it up after the fact.

How it is packaged into the image

The `alfresco` service's Dockerfile (inline, in the compose file) layers two files from the `docker-compose` folder on top of the base governance repository image:

```
FROM quay.io/alfresco/alfresco-governance-repository-enterprise:26.3.0-A.20
COPY behaviour-derive-names-context.xml \
    /usr/local/tomcat/shared/classes/alfresco/extension/behaviour-derive-names-context.xml
COPY derive-names-onCreateNode.js \
    /usr/local/tomcat/shared/classes/alfresco/extension/scripts/derive-names-onCreateNode.js
```

Any `*-context.xml` file dropped into `shared/classes/alfresco/extension` is picked up automatically by Alfresco's Spring context scanning at startup — no WAR rebuild or module packaging (AMP/JAR) is required for this kind of simple extension.

behaviour-derive-names-context.xml

Registers a JavaScript-backed policy behaviour on the `{http://www.alfresco.org/model/content/1.0}person` type, bound to the `onCreateNode` policy, pointing at the script below.

derive-names-onCreateNode.js

Fires once, immediately after a new `cm:person` node is created (including by LDAP sync). Logic:

- If `firstName` is blank/whitespace-only and `lastName` is populated → copy `lastName` into `firstName`.
- If `lastName` is blank and `firstName` is populated → copy it the other way.
- If **both** are blank → default both to the `userName`.
- Every change is written back with `person.save()` and logged, e.g. `[derive-names] jonly: blank lastName derived from firstName 'Jon'`.

6. Step-by-Step: Running It Yourself

- 1 Open a terminal (Git Bash, PowerShell, or WSL) and change into `C:\AlfrescoWork\acs-deployment\docker-compose`.
- 2 Make sure Docker Desktop is running and has at least ~13 GB of memory allocated (this stack includes the full Governance/Search/Transform set of services, not just LDAP).
- 3 Start the stack in the background: `docker compose -f 26.N-compose-governance.yaml up -d`
- 4 Watch the `openldap` container become healthy first (`docker compose -f 26.N-compose-governance.yaml ps`); the `alfresco` container will not start until it is, because of the `depends_on: openldap: condition: service_healthy` dependency.
- 5 Tail the repository log until Tomcat/Alfresco finishes starting: `docker compose -f 26.N-compose-governance.yaml logs -f alfresco`
- 6 Trigger (or wait for) the scheduled LDAP synchronization job so the seed users are imported as Alfresco persons/authorities. It also runs automatically a short time after the repository starts.
- 7 Log in to Share (`http://localhost:8080/share`) as `jdoe / password123`, `jonly / password123`, or `smithonly / password123` to confirm LDAP authentication works end-to-end, and inspect each user's profile to confirm first/last name were derived correctly for the edge-case accounts.
- 8 The built-in `admin / admin` account still works via the `alfrescoNTLm` fallback in the authentication chain, since it does not exist in LDAP.

Direct LDAP queries (bypassing Alfresco, useful for isolating problems)

```
# From the host (port 389 is published) or via "docker compose exec openldap ..."
ldapsearch -x -H ldap://localhost:389 \
  -D "cn=admin,dc=example,dc=org" -w admin \
  -b "dc=example,dc=org" "(uid=jdoe)"

# Test a simple bind as an end user (authentication path)
ldapwhoami -x -H ldap://localhost:389 \
  -D "uid=jdoe,ou=users,dc=example,dc=org" -w password123
```

7. Adding or Changing Test Users

- Edit `docker-compose/ldap-custom-ldif/50-users.ldif` and add further `dn: uid=<name>,ou=users,dc=example,dc=org` entries following the same `inetOrgPerson` pattern.
- Bootstrap LDIFs under `osixia/openldap` are only applied on a **fresh** database. If the `openldap` container has already initialized once, either: (a) remove its container and any named volume so it reinitializes from the LDIF, or (b) add the new entries live with `ldapadd` against the running server using the admin credentials above.
- After adding users, either wait for the next scheduled LDAP synchronization run or trigger one manually from the Alfresco Admin Console (`/alfresco/s/enterprise/admin/admin-ldap` or the equivalent Synchronization page) so the new accounts appear as Alfresco persons.
- To add groups, create entries with `objectClass: groupOfNames` under the `groupSearchBase` (`dc=example,dc=org`), with a `cn` attribute (group id) and one or more `member` attributes holding member user DNS, matching `ldap.synchronization.groupType / groupMemberAttributeName`.

8. Troubleshooting

Symptom	Likely cause / fix
alfresco container never starts	openldap healthcheck failing. Check `docker compose logs openldap`; confirm port 389 isn't already in use
LDIF changes don't appear	osixia/openldap only applies bootstrap LDIFs on first init. Remove the container + its data volume (`docker volume rm \$(docker volume ls -q --filter=driver=local --filter=name=openldap-data)`) and re-run the compose
User can't log in via LDAP	Verify `userNameFormat` matches the real DN pattern; test the bind directly with `ldapwhoami` as shown in the docs
User logs in but has no profile / isn't visible in Share	Synchronization subsystem is separate from authentication - confirm `ldap.synchronization.active=true` in the config
Synced user has a blank First Name or Last Name	Expected for the `jonly` / `smithonly` seed accounts before the derive-names behaviour runs; confirm by checking the LDAP entry
Need to start over completely	`docker compose -f 26.N-compose-governance.yaml down -v` removes containers and volumes (including data)

9. Quick Reference Card

Setting	Value
Compose file	docker-compose/26.N-compose-governance.yaml
LDAP image	osixia/openldap:1.5.0
LDAP host:port (from Alfresco)	openldap:389
LDAP host:port (from host machine)	localhost:389
Base DN	dc=example,dc=org
User search base	ou=users,dc=example,dc=org
Group search base	dc=example,dc=org
Admin bind DN / password	cn=admin,dc=example,dc=org / admin
User DN pattern	uid=%s,ou=users,dc=example,dc=org
Test users / password	jonly, smithonly, jdoe / password123
Repository fallback login	admin / admin (via alfrescoNtlm chain entry)
Share URL	http://localhost:8080/share
Seed data file	docker-compose/ldap-custom-ldif/50-users.ldif
Name-fixup extension files	behaviour-derive-names-context.xml, derive-names-onCreateNode.js